FINAL PROJECT
MINI PROCESSOR (SINGLE-CYCLE CPU)

1.Instruction Set Architecture (ISA)

All instructions are 16 bits wide. Bits [15:14] select the instruction class.

1.1 R-Type - register/register ALU operation

15 14	13 12 11 10	9 8	7 6	5 4	3 2 1 0
00	ALU op	rd	rs1	rs2	reserved (0)

`alu_op` reuses the exact 16-entry table from Assignment 2 (0000 = ADD ... 1111 = XNOR). Example: `ADD R2, R0, R1` performs *R2 <= R0 + R1*.

1.2 I-Type - load immediate

15 14	13 12	11 10 9 8 7 6 5 4	3 2 1 0
01	rd	imm[7:0]	reserved (0)

`LOADI rd, imm` - loads an 8-bit immediate (zero-extended) directly into rd. Used to initialize registers, since there is no other way to get a constant into the datapath.

1.3 (Modified) Branch / Jump

15 14	13 12	11 10	9 8	7 6	5 4 3 2 1 0	
10	00	unused / reserved (0)			target [5:0]	**JMP target**
10	01	rs1	rs2	reserved (0)	target [5:0]	**BEQ rs1, rs2, target**
10	10	rs1	rs2	reserved (0)	target [5:0]	**BNE rs1, rs2, target**

JMP unconditionally sets *PC = target*. BEQ compares rs1 and rs2 through the ALU (subtract, check zero flag) and branches only if they are equal; otherwise PC advances normally.

BNE (Bonus) compares rs1 and rs2 through the ALU (subtract, check zero flag) and branches only if they are **not equal** (zero flag = 0); otherwise PC advances normally.

1.4 System

15 14 13 12 11 ... 0

11	00	unused (reserved for bonus opcodes)	**HALT**
11	01	unused	**NOP**

HALT freezes the PC (pc_src = 11) so the program stops cleanly instead of running off the end of memory. NOP does nothing for one cycle.

1.5 Opcode Summary Table

Class	Bits [15:14]	Mnemonic(s)	Purpose
R-Type	00	ADD, SUB, AND, OR, XOR, NOT, INC, SHIFT, ROTATE, NAND, NOR, XNOR ...	Register-register ALU ops (Assignment 2 table)
I-Type	01	LOADI	Load an 8-bit constant into a register
Branch/Jump	10	JMP, BEQ, BNE	Unconditional / conditional control transfer
System	11	HALT, NOP	Stop execution / do nothing

2. Instruction Set

```
    initial begin
        // addr : instruction                       : description & verification check
        mem[0]  = 16'b0100_0000_0101_0000; // LOADI R0, 5                -> R0 = 5
        mem[1]  = 16'b0101_0000_0011_0000; // LOADI R1, 3                -> R1 = 3
        mem[2]  = 16'b0000_0010_0001_0000; // ADD   R2, R0, R1           -> R2 = 8
        mem[3]  = 16'b0000_0111_0001_0000; // SUB   R3, R0, R1           -> R3 = 2
        mem[4]  = 16'b1001_1011_0000_1110; // BEQ   R2, R3, done (tgt=14)-> NOT TAKEN (8 != 2) -> advances to 5
        mem[5]  = 16'b0011_1011_0001_0000; // XOR   R3, R0, R1           -> R3 = 5 ^ 3 = 6 (Logic)
        mem[6]  = 16'b1010_0001_0000_1000; // BNE   R0, R1, skip1 (tgt=8)-> IS TAKEN (5 != 3) -> jumps to 8
        mem[7]  = 16'b0100_0110_0011_0000; // LOADI R0, 99               -> SKIPPED
        mem[8]  = 16'b1010_0101_0000_1110; // skip1: BNE R1, R1, done   -> BNE NOT TAKEN (3 != 3 is false) -> advances to 9
        mem[9]  = 16'b1001_0000_0000_1011; // BEQ   R0, R0, skip2 (tgt=11)-> BEQ TAKEN (5 == 5 is true) -> jumps to 11
        mem[10] = 16'b0101_0110_0011_0000; // LOADI R1, 99               -> SKIPPED
        mem[11] = 16'b0000_1100_0110_0000; // skip2: MUL R0, R1, R2     -> R0 = 3 * 8 = 24
        mem[12] = 16'b1000_0000_0000_1110; // JMP   done (tgt=14)        -> JMP to 14
        mem[13] = 16'b0110_0110_0011_0000; // LOADI R2, 99               -> SKIPPED (Trap)
        mem[14] = 16'b1100_0000_0000_0000; // done: HALT                 -> Freezes PC at 14
    end
//we expect at the end R0=24, R1=3, R2=8, R3=6
```

3.Testbench

3.1 Monitor Output

```
#
# Time=40000 | PC=0 | Instr=0100000001010000 | Zero=1 | RegWrite=1
# Time=50000 | PC=1 | Instr=0101000000110000 | Zero=0 | RegWrite=1
# Time=70000 | PC=2 | Instr=0000001000010000 | Zero=0 | RegWrite=1
# Time=90000 | PC=3 | Instr=0000011100010000 | Zero=0 | RegWrite=1
# Time=110000 | PC=4 | Instr=1001101100001110 | Zero=0 | RegWrite=0
# Time=130000 | PC=5 | Instr=0011101100010000 | Zero=0 | RegWrite=1
# Time=150000 | PC=6 | Instr=1010000100001000 | Zero=0 | RegWrite=0
# Time=170000 | PC=8 | Instr=1010010100001110 | Zero=1 | RegWrite=0
# Time=190000 | PC=9 | Instr=1001000000001011 | Zero=1 | RegWrite=0
# Time=210000 | PC=11 | Instr=0000110001100000 | Zero=0 | RegWrite=1
# Time=230000 | PC=12 | Instr=1000000000001110 | Zero=0 | RegWrite=0
# Time=250000 | PC=14 | Instr=1100000000000000 | Zero=0 | RegWrite=0
#
# ================ FINAL REGISTER VALUES ================
# R0 = 24 (Expected: 24)
# R1 = 3 (Expected: 3)
# R2 = 8 (Expected: 8)
# R3 = 6 (Expected: 6)
# Final PC = 14 (Expected: 14)
# ========================================================
#
```

3.2 Waveform

